Cricket Player Performance Tracking Application

Salesforce-Based Project Documentation

Team ID: SWTID-2026-7186
Team Size: 5
Team Leader: Anitha D
Team member: Sandhiya K
Team member: Deepika Sakthi
Team member: Vanmathi S
Team member: Sujitha Sundaravel

Github link:

1. INTRODUCTION

1.1 Project Overview

CRICKSTATS is a Salesforce-based application designed to track, manage, and analyze cricket players' performance on a match-by-match basis. The system uses custom objects, relationships, dashboards, reports, and automation tools to provide structured performance insights. It enables coaches, analysts, team managers, and Salesforce administrators to record player statistics, generate reports, and support strategic decision-making and player development.

1.2 Purpose

The main purpose of CRICKSTATS is to digitize player performance records, improve the accuracy and accessibility of cricket statistics, provide timely analytics, support player development and team planning, and automate data management using Salesforce.

2. IDEATION PHASE

2.1 Problem Statement

A cricket team coach needs to track player performance across matches but may not have accurate statistics readily available because data is stored in different files and formats. This can make team-selection decisions difficult. A team manager needs to maintain complete player records, but manual updates and verification take considerable time. A sports analyst needs to analyze trends and compare performances, but scattered data makes centralized analysis difficult.

2.2 Empathy Map Canvas

Target users include Team Managers, Coaches, Analysts, and Salesforce Administrators. Users think and feel that they need accurate data and better insights. They hear feedback from management, see multiple data sources, and often collect statistics manually. Their major pains are data errors and delays, while expected gains include faster decisions and automation.

2.3 Brainstorming

Ideas considered included a web-based player management system, mobile statistics tracker, Salesforce-integrated CRM, AI-based performance prediction, and dashboard-based analytics. The selected idea was a Salesforce-based cricket performance tracking platform named CRICKSTATS because Salesforce provides structured data management, security, automation, reporting, and dashboard capabilities.

3. REQUIREMENT ANALYSIS PHASE

3.1 Customer Journey Map

The main user journey is: Login → Enter Player Data → Record Match Performance → Generate Reports → Analyze Results → Decision Making.

3.2 Functional Requirements

FR-1: Player Registration. FR-2: Match Performance Entry. FR-3: Dashboard Generation. FR-4: Report Creation. FR-5: Flow Automation.

3.3 Non-Functional Requirements

Usability: The system should provide simple navigation, intuitive forms, and readable dashboards. Security: role-based access, secure authentication, controlled permissions, and protected player data should be used. Reliability: data should be stored accurately with suitable error handling and recovery. Performance: pages, reports, and dashboards should respond efficiently. Availability: the cloud application should be accessible when required with minimal downtime. Scalability: the design should support increasing numbers of players, matches, teams, and users.

3.4 Data Flow Diagram

User → Salesforce Lightning UI → Player Object → Match Performance Object → Reports → Dashboard.

3.5 Technology Stack

Frontend: Salesforce Lightning. Backend/Application Logic: Salesforce Objects and Flows. Database: Salesforce Cloud. Analytics: Salesforce Reports and Dashboards. Automation: Salesforce Flow

4. SPROJECT DESIGN PHASE

4.1 Problem–Solution Fit

Traditional cricket performance records can be manual, scattered, and difficult to analyze. CRICKSTATS addresses these issues through centralized Salesforce records, relationships between Player and Match Performance, automated calculations, reports, dashboards, and flows. This creates a structured environment for storing and retrieving performance information.

4.2 Proposed Solution

The proposed solution includes custom Player and Match Performance objects, a master-detail relationship, lookup capability where appropriate, roll-up summary fields, a dedicated Lightning App, automated flows, reports, and interactive dashboards. Key features include centralized data, live statistics, secure access, scalable design, and simplified reporting.

4.3 Solution Architecture

Presentation Layer: Salesforce Lightning UI. Application Layer: Salesforce Flow and validation logic. Data Layer: Player and Match Performance objects. Analytics Layer: Reports and Dashboards.

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

The project follows an Agile-based approach. Work is organized into sprints covering environment setup, data modeling, navigation, UI design, field configuration, validation, automation, reporting, and testing.

5.2 Product Backlog

USN-1: Create a Salesforce developer account — 1 point — High. USN-2: Create Player and Match Performance objects — 3 points — High. USN-3: Create separate tabs — 3 points — High. USN-4: Create a dedicated Lightning App — 3 points — High. USN-5: Create player and match fields — 5 points — Medium. USN-6: Customize page layouts — 5 points — High. USN-7: Validate player and match inputs — 3 points — High. USN-8: Create automated flows to update statistics — 5 points — High.

5.3 Sprint Schedule

Sprint-1: 20 story points, 6 days, 24 Oct 2022–29 Oct 2022. Sprint-2: 20 story points, 6 days, 31 Oct 2022–05 Nov 2022. Sprint-3: 20 story points, 6 days, 07 Nov 2022–12 Nov 2022. Sprint-4: 20 story points, 6 days, 14 Nov 2022–19 Nov 2022.

6. PROJECT DEVELOPMENT PHASE

6.1 Project Flow

Milestone 1: Developer Account Creation. Milestone 2: Object Creation. Milestone 3: Tabs. Milestone 4: Lightning App. Milestone 5: Fields and Relationships. Milestone 6: Roll-up Summary. Milestone 7: Flows.

6.2 Milestone 1 – Salesforce Developer Account

CRICKSTATS – Cricket Player Performance Tracking | Page 3Create a Salesforce Developer account through the official Salesforce Developer signup page. Enter the required personal, role, organization, country, postal code, and username information. Activate the account through the verification email, set a password, and access Salesforce Setup.

6.3 Milestone 2 – Object Creation

Create a custom Player object with Player Name as the record name. Enable suitable options such as reporting, activities, field history tracking, and search. Create a Match Performance custom object in the same way. The two objects form the core data model of CRICKSTATS.

6.4 Milestone 3 – Tabs

Create custom tabs for Player and Match Performance. Select an appropriate tab style, retain the required profile settings, and make the tabs available for the application.

6.5 Milestone 4 – Lightning App

Create a Lightning App named CRICKSTATS. Use Standard navigation and add Players, Match Performances, Reports, and Dashboards as navigation items. Assign the application to the required user profile, such as System Administrator.

6.6 Milestone 5 – Player Fields

Create Player fields such as Player Name, Player Number, Profile Picture, Date of Birth, Role, Batting Style, Bowling Style, Nationality, Team, and AVG Score. Suggested role values include Batsman, Bowler, Batsman & Wicket Keeper, and All Rounder. Batting style can include Right-Handed and Left-Handed. Bowling style can include Right-Arm Fast, Left-Arm Fast, Right-Arm Medium, Left-Arm Medium, Right-Arm Spin, and Left-Arm Spin. Nationality and Team can be configured as picklists according to project requirements.

6.7 Milestone 5 – Match Performance Fields

Create Match Performance fields such as Match Performance Name, Player of the Match, Match Result, Runs Scored, Balls Faced, Fours, Sixes, Overs Bowled, Wickets Taken, Runs Conceded, Maiden Overs, Stumpings, and Catches. Create a Master-Detail Relationship from Match Performance to Player so each performance record is associated with its parent player.

6.8 Milestone 6 – Roll-up Summary

On Player, create roll-up summary fields using Match Performances: Matches Played (COUNT), Number of 50s and Number of 100s (COUNT with suitable filters), Total 4s (SUM of Fours), Total 6s (SUM of Sixes), Total Runs (SUM of Runs Scored), Total Overs (SUM of Overs Bowled), Total Wickets (SUM of Wickets Taken), Total Stumpings (SUM of Stumpings), and Total Catches (SUM of Catches).

6.9 Milestone 7 – Screen Flow

Create a Screen Flow named Player Flow. Add a screen containing a Team picklist using a Picklist Choice Set from Player.Team. Get Player records filtered by the selected team. Display the resulting records in a Data Table with columns such as Profile Picture, Player Name, Total Runs, AVG Score, Batting Style, Bowling Style, Matches Played, Number of 50s, and Number of 100s. Save and activate the flow.

6.10 Flow on Home Page

Add the Player Flow to the CRICKSTATS Home Page using the standard Flow component. Save and activate the page, assign it to the CRICKSTATS application, and verify that users can select a team and view player information.

7. FUNCTIONAL & PERFORMANCE TESTING

7.1 Functional Testing

Testing should verify Player and Match Performance record creation, field validation, relationship behavior, roll-up summary calculations, report generation, dashboard visualization, and flow execution. Test records should be entered CRICKSTATS – Cricket Player Performance Tracking | Page 4before checking calculated results.

7.2 Performance Testing

Evaluate response time during data entry, report generation, and dashboard loading. Test multiple user interactions and verify that the application remains stable and that displayed statistics are consistent with stored records.

7.3 Test Cases

TC-01: Create a Player record and verify that all required fields accept valid values. TC-02: Create a Match Performance record and associate it with a Player. TC-03: Enter runs, fours, and sixes and verify roll-up totals. TC-04: Enter wickets and verify Total Wickets. TC-05: Select a team in Player Flow and verify that matching players are displayed. TC-06: Generate a report and compare values against source records.

8. ADVANTAGES & DISADVANTAGES

8.1 Advantages

Centralized system; real-time reporting and analytics; secure Salesforce platform; scalable cloud-based design; reduced manual work; relationship-based data integrity; automated calculations.

8.2 Disadvantages

Salesforce licensing or edition constraints may apply; internet connectivity is generally required for cloud access; offline functionality is limited; advanced analytics or integrations may require additional configurat

9. CONCLUSION

9.1 Conclusion

CRICKSTATS demonstrates a practical Salesforce solution for cricket player performance tracking. By combining custom objects, relationships, roll-up summaries, flows, reports, dashboards, and Lightning App capabilities, the system provides a structured way to store match-by-match statistics and present useful performance information. The project also demonstrates practical application of SDLC and Agile development concepts in a CRM-based cloud environment.

Program:

cosplayers=[

{id:1,name:'Ruturaj Gaikwad',number:31,state:'Maharashtra',nationality:'India',role:'Batsman',batting:'Right-Handed',bowling:'Right-Arm Medium',team:'Chennai Super Kings',matches:18,runs:742,average:46.38,fours:68,sixes:28,wickets:0,catches:9,image:'https://i.pravatar.cc/100?img=11'}, {id:2,name:'Jasprit Bumrah',number:93,state:'Gujarat',nationality:'India',role:'Bowler',batting:'Right-Handed',bowling:'Right-Arm Fast',team:'Mumbai Indians',matches:16,runs:48,average:12,wickets:27,fours:3,sixes:1,catches:8,image:'https://i.pravatar.cc/100?img=12'}, {id:3,name:'Sanju Samson',number:9,state:'Kerala',nationality:'India',role:'Batsman & Wicket Keeper',batting:'Right-Handed',bowling:'Right-Arm Medium',team:'Rajasthan Royals',matches:17,runs:681,average:45.4,fours:54,sixes:31,wickets:0,catches:14,image:'https://i.pravatar.cc/100?img=13'}, {id:4,name:'Ravindra Jadeja',number:8,state:'Gujarat',nationality:'India',role:'All Rounder',batting:'Left-Handed',bowling:'Left-Arm Spin',team:'Chennai Super Kings',matches:18,runs:414,average:34.5,fours:32,sixes:17,wickets:19,catches:11,image:'https://i.pravatar.cc/100?img=14'}, {id:5,name:'Virat Kohli',number:18,state:'Delhi',nationality:'India',role:'Batsman',batting:'Right-Handed',bowling:'Right-Arm Medium',team:'Royal Challengers Bangalore',matches:18,runs:741,average:49.4,fours:68,sixes:24,wickets:0,catches:10,image:'https://i.pravatar.cc/100?img=15'}, {id:6,name:'Kuldeep Yadav',number:23,state:'Uttar Pradesh',nationality:'India',role:'Bowler',batting:'Left-Handed',bowling:'Left-Arm Spin',team:'Delhi Capitals',matches:15,runs:31,average:10.3,fours:2,sixes:0,wickets:22,catches:6,image:'https://i.pravatar.cc/100?img=16'},]; const performances = [{player:'Ruturaj Gaikwad',team:'Chennai Super Kings',runs:78,balls:52,fours:8,sixes:3,overs:0,wickets:0,result:'WON',date:'18 Sep 2025'}, {player:'Jasprit Bumrah',team:'Mumbai Indians',runs:4,balls:6,fours:0,sixes:0,overs:4,wickets:4,result:'WON',date:'17 Sep 2025'}, {player:'Sanju Samson',team:'Rajasthan Royals',runs:104,balls:61,fours:9,sixes:5,overs:0,wickets:0,result:'WON',date:'16 Sep 2025'}, {player:'Virat Kohli',team:'Royal Challengers Bangalore',runs:91,balls:57,fours:7,sixes:4,overs:0,wickets:0,result:'LOST',date:'15 Sep 2025'}, {player:'Ravindra Jadeja',team:'Chennai Super Kings',runs:42,balls:27,fours:3,sixes:2,overs:4,wickets:2,result:'WON',date:'14 Sep 2025'}]; const teams = ['Chennai Super Kings','Delhi Capitals','Kolkata Knight Riders','Mumbai Indians','Punjab Kings','Rajasthan Royals','Royal Challengers Bangalore','Sunrisers

```
Hyderabad','Gujarat Titans','Lucknow Super Giants']; let currentView = 'dashboard'; let selectedReport = ''; const appView = document.getElementById('appView'); const avatar = p => <img class="player-avatar" src="${p.image}" alt="${p.name}">; const viewHeader = (eyebrow,title,subtitle,button='') => <div class="page-heading"><div><p class="eyebrow">${eyebrow}</p><h1 class="page-title">${title}</h1><p class="page-subtitle">${subtitle}</p></div>${button}</div>; function dashboard(){ const top = [...players].sort((a,b)=>b.runs-a.runs).slice(0,4); return ${viewHeader('Thursday, 24 September 2026','Good morning','Here is what is happening across your squads today.','<button class="primary-btn" data-action="add-performance">+ Add performance</button>')} <div class="stat-grid"><div class="stat-card"><div class="stat-top">Total players <span class="stat-icon">♙</span></div><div class="stat-value">${players.length}</div><span class="trend">↑ 12% <small>vs last season</small></span></div><div class="stat-card"><div class="stat-top">Total matches <span class="stat-icon">◉</span></div><div class="stat-value">86</div><span class="trend">↑ 8% <small>vs last season</small></span></div><div class="stat-card"><div class="stat-top">Total runs <span class="stat-icon">↗</span></div><div class="stat-value">3,248</div><span class="trend">↑ 18% <small>vs last season</small></span></div><div class="stat-card"><div class="stat-top">Total wickets <span class="stat-icon">✦</span></div><div class="stat-value">142</div><span class="trend">↑ 6% <small>vs last season</small></span></div></div> <div class="dashboard-grid"><div class="panel chart-panel"><div class="panel-head"><div><h2 class="panel-title">Performance overview</h2><span class="panel-note">Runs and wickets across the season</span></div><select class="select-input"><option>Last 6 months</option><option>This season</option></select></div><div class="bar-chart">${[62,78,55,84,72,91].map((n,i)=>

${['Apr','May','Jun','Jul','Aug','Sep'][i]}
).join('')}</div><div class="legend"><span>Runs scored</span><span>Wickets taken</span></div></div><div class="panel"><div class="panel-head"><div><h2 class="panel-title">Top performers</h2><span class="panel-note">This season</span></div><button class="icon-button" data-view="analytics">→</button></div><div class="performer-list">${top.map(p=>
```

```
${avatar(p)}
**${p.name}**${p.team.replace('Royal Challengers Bangalore','RCB')}
**${p.runs}**runs
).join('')}</div></div></div> <div class="insight-row"><div
class="panel"><div class="panel-head"><div><h2 class="panel-
title">Quick insights</h2><span class="panel-note">Squad
snapshot</span></div></div><div class="mini-stat"><span>Highest run
scorer</span><b>Ruturaj Gaikwad</b></div><div class="mini-
stat"><span>Highest wicket taker</span><b>Jasprit Bumrah</b></div><div
class="mini-stat"><span>Best average</span><b>Virat Kohli ·
49.4</b></div><div class="mini-stat"><span>Player of the
match</span><b>Sanju Samson · 4</b></div></div><div class="panel"><div
class="panel-head"><div><h2 class="panel-title">Latest
performances</h2><span class="panel-note">Most recent
entries</span></div><button class="icon-button" data-
view="performances">→</button></div><div class="performer-
list">${performances.slice(0,3).map(m=>

**${m.player}**${m.date} · ${m.result}
**${m.runs}**runs
).join('')}</div></div></div>; } function playersView(){return
${viewHeader('Squad directory','Players','Manage player profiles and
track individual career performance.','<button class="primary-btn"
data-action="add-player">+ Add player</button>')}<div
class="toolbar"><input class="search-input" id="playerSearch"
placeholder="⌕ Search players..."/><select class="select-input"
id="teamFilter"><option value="">All teams</option>${teams.map(t=>

${t}

).join('')}</select><select class="select-input"
id="roleFilter"><option value="">All
roles</option><option>Batsman</option><option>Bowler</option><option>A
ll Rounder</option></select></div><div class="panel table-panel"><div
class="table-wrap"><table class="data-
table"><thead><tr><th>Player</th><th>Team</th><th>Role</th><th>Matches
</th><th>Runs</th><th>Average</th><th>Wickets</th></tr></thead><tbody
id="playerRows">${playerRows(players)}</tbody></table></div></div>}
function playerRows(list){return list.length?list.map(p=><tr><td><div class="player-
cell">${avatar(p)}<div><b>${p.name}</b><span>#${p.number} ·
```

```
${p.state}</span></div></div></td><td>${p.team.replace('Royal
Challengers Bangalore','RCB')}</td><td><span
class="tag">${p.role}</span></td><td>${p.matches}</td><td><b>${p.runs}
</b></td><td>${p.average}</td><td>${p.wickets}</td></tr>).join(''):<tr><td
colspan="7"><div class="empty-state">No players match your
filters.</div></td></tr>} function performancesView(){return
${viewHeader('Match log','Match performance','Every innings, spell,
catch and stumping in one place.','<button class="primary-btn" data-
action="add-performance">+ Add performance</button>')}<div
class="toolbar"><input class="search-input" id="performanceSearch"
placeholder="⌕ Search player or team..."/><select class="select-
input"><option>All
results</option><option>WON</option><option>LOST</option><option>DRAW<
/option></select></div><div class="panel table-panel"><div
class="table-wrap"><table class="data-
table"><thead><tr><th>Player</th><th>Date</th><th>Runs /
balls</th><th>4s /
6s</th><th>Overs</th><th>Wickets</th><th>Result</th></tr></thead><tbod
y
id="performanceRows">${performanceRows(performances)}</tbody></table><
/div></div>} function performanceRows(list){return list.map(m=><tr><td><div
class="player-cell"><div class="avatar"
style="width:30px;height:30px">${m.player.split('
').map(x=>x[0]).join('').slice(0,2)}</div><div><b>${m.player}</b><span
>${m.team}</span></div></div></td><td>${m.date}</td><td><b>${m.runs}</
b> / ${m.balls}</td><td>${m.fours} /
${m.sixes}</td><td>${m.overs}</td><td>${m.wickets}</td><td><span
class="tag">${m.result}</span></td></tr>).join('')} function teamsView(){return
${viewHeader('Competition map','Teams','Explore player depth and
performance by team.')}<div class="team-grid">${teams.map((t,i)=>
```

${t.split(' ').map(x=>x[0]).join('').slice(0,2)}
**${t}**${players.filter(p=>p.team===t).length || (i%3)+1} registered players · View squad →
).join('')} } function reportsView(){const reports=[['🏏','Player Performance Report','A complete view of player output across batting, bowling and fielding.'],['↗','Batting Report','Compare runs, averages, boundaries and strike contributions.'],['✦','Bowling Report','Review overs, economy, wickets and maiden overs.'],['◆','Team Report','Team-wise performance and squad depth.'],['◉','Match Report','Match-by-match performance log with result

filters.'],['◊','Fielding Report','Catches and stumpings by player and team.']];return ${viewHeader('Reporting suite','Reports','Ready-to-use performance reports for coaches and analysts.')}<div class="report-grid">${reports.map(r=>

${r[0]}
**${r[1]}**${r[2]}→
).join('')} } function reportDetailView(){const title=selectedReport||'Player Performance Report';const rows=players.map(p=><tr><td> ${avatar(p)}<div>${p.name}${p.team}</div></div></td><td>${p.matches}</td><td>${p.runs}</td><td>${p.average}</td><td>${p.fours}</td><td>${p.sixes}</td><td>${p.wickets}</td></tr>).join('');return ${viewHeader('Report detail',title,'Detailed performance data for coaches and analysts.','<button class="secondary-btn" data-action="back-reports">← Back to reports</button>')}<div class="panel table-panel"><div class="panel-head"><div><h2 class="panel-title">${title}</h2>Current squad data <button class="primary-btn" data-action="export-report">Export report</button> <table class="data-table"><thead><tr><th>Player</th><th>Matches</th><th>Total runs</th><th>Average</th><th>4s</th><th>6s</th><th>Wickets</th></tr></thead><tbody>${rows}</tbody></table></div></div>} function analyticsView(){const runRanks=[...players].sort((a,b)=>b.runs-a.runs), wicketRanks=[...players].sort((a,b)=>b.wickets-a.wickets);const list=(items,key,unit)=>items.slice(0,5).map((p,i)=><div class="rank-item"><div class="rank-number">0${i+1} ${avatar(p)}<div class="rank-name">${p.name}${p.team}</div><div class="rank-score">${p[key]} <small>${unit}</small>).join('');return ${viewHeader('Decision center','Analytics','Find the players shaping the season.')}<div class="analytics-grid"><div class="panel"><div class="panel-head"><div><h2 class="panel-title">Highest run scorers</h2>All teams · 2025 season</div></div><div class="rank-list">${list(runRanks,'runs','runs')} <h2 class="panel-title">Highest wicket takers</h2>All teams · 2025 season ${list(wicketRanks,'wickets','wkts')} } function adminView(){return ${viewHeader('Workspace controls','Admin','Manage

```
access and keep your performance workspace consistent.')}<div
class="analytics-grid"><div class="panel"><div class="panel-head"><h2
class="panel-title">Access roles</h2></div><div class="mini-
stat"><span>Administrators</span><b>2 users</b></div><div class="mini-
stat"><span>Coaches</span><b>6 users</b></div><div class="mini-
stat"><span>Analysts</span><b>4 users</b></div><div class="mini-
stat"><span>Last permission review</span><b>12 Sep
2025</b></div></div><div class="panel"><div class="panel-head"><h2
class="panel-title">Data quality</h2></div><div class="mini-
stat"><span>Complete player profiles</span><b>98%</b></div><div
class="mini-stat"><span>Performances this season</span><b>86
records</b></div><div class="mini-stat"><span>Pending
review</span><b>3 records</b></div><div class="mini-stat"><span>System
status</span><b style="color:var(--
green)">Healthy</b></div></div></div>} function
render(view=currentView){currentView=view;const
pages={dashboard,players:playersView,performances:performancesView,teams:teamsView,repo
rts:reportsView,reportDetail:reportDetailView,analytics:analyticsView,admin:adminView};appV
iew.innerHTML=pagesview;document.getElementById('breadcrumbCurrent').textContent=view
==='dashboard'?'Dashboard':view==='reportDetail'?'Report
detail':view[0].toUpperCase()+view.slice(1);document.querySelectorAll('.nav-
item').forEach(n=>n.classList.toggle('active',n.dataset.view===view));bindViewEvents()}
function modal(type){const
isPlayer=type==='player';document.getElementById('modalRoot').innerHTML=<div
class="modal-backdrop" id="modalBackdrop"><div class="modal"><div
class="modal-head"><div><h2>${isPlayer?'Add player':'Add match
performance'}</h2><p>${isPlayer?'Create a complete player profile for
your squad.':'Record a player performance from the latest
match.'}</p></div><button class="close-btn" data-action="close-
modal">×</button></div><form id="entryForm"><div class="form-
grid">${isPlayer?
```

Player name
Player number
State
Tamil Nadu
Kerala
Karnataka
Andhra Pradesh

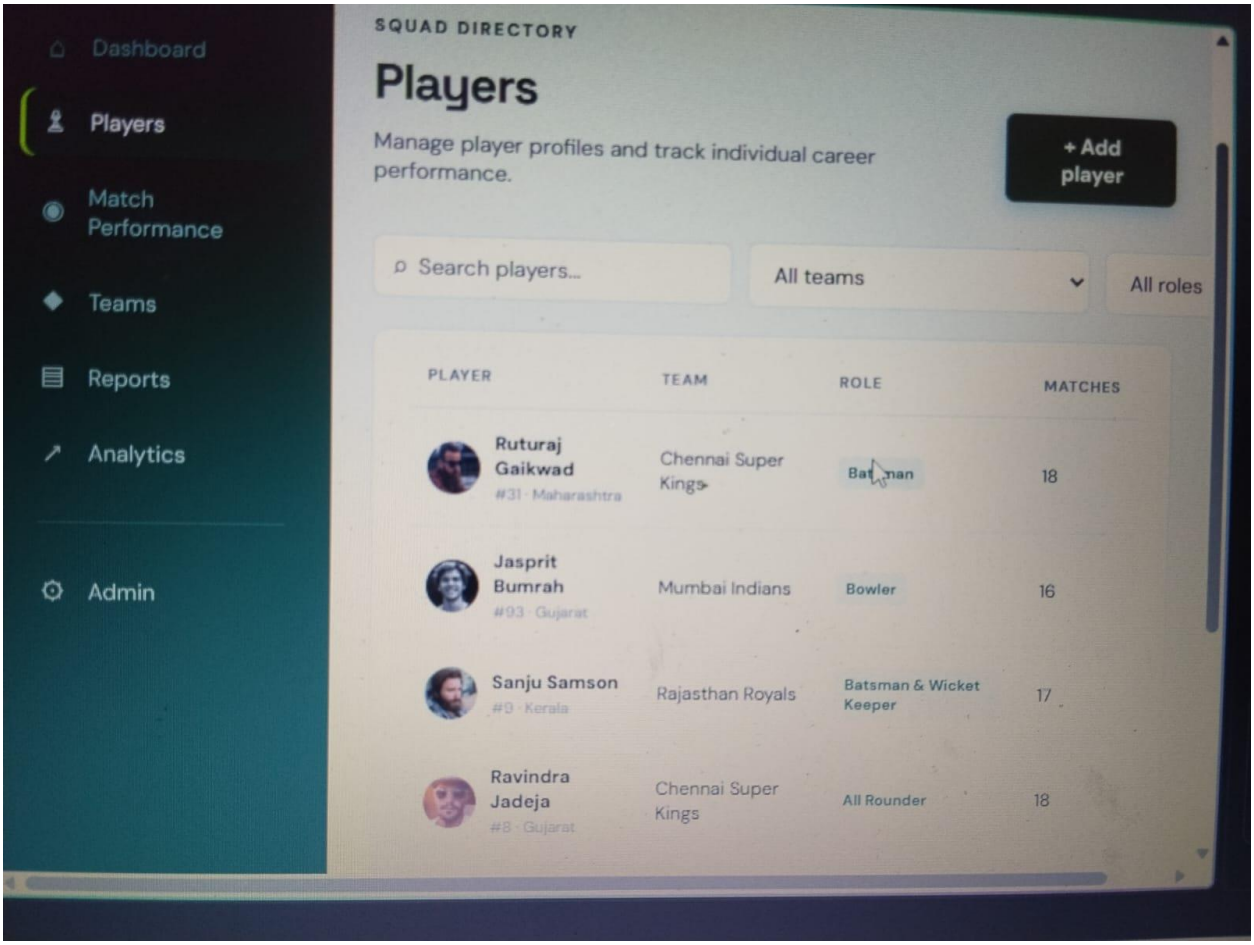
Telangana
Maharashtra
Gujarat
Rajasthan
Punjab
Haryana
Delhi
Uttar Pradesh
West Bengal
Odisha
Bihar
Role
Batsman
Bowler
Batsman & Wicket Keeper
All Rounder
Team
${teams.map(t=><option>${t}</option>).join('')}
Batting style
Right-Handed
Left-Handed
Bowling style
Right-Arm Fast
Left-Arm Fast
Right-Arm Medium
Left-Arm Medium
Right-Arm Spin
Left-Arm Spin
:

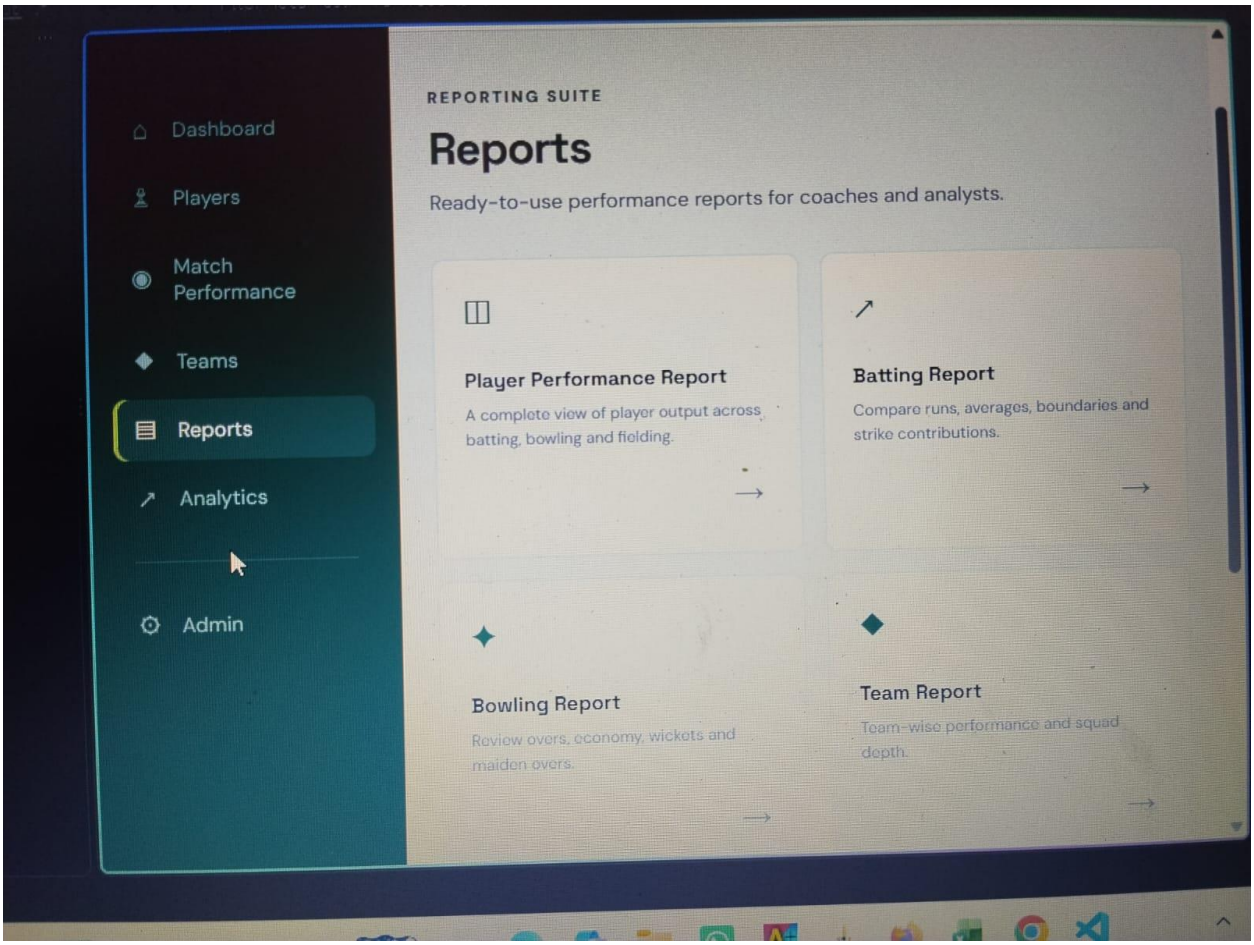
Player
${players.map(p=><option>${p.name}</option>).join('')}
Runs scored
Balls faced
Fours
Sixes
Overs bowled
Wickets taken
Match result

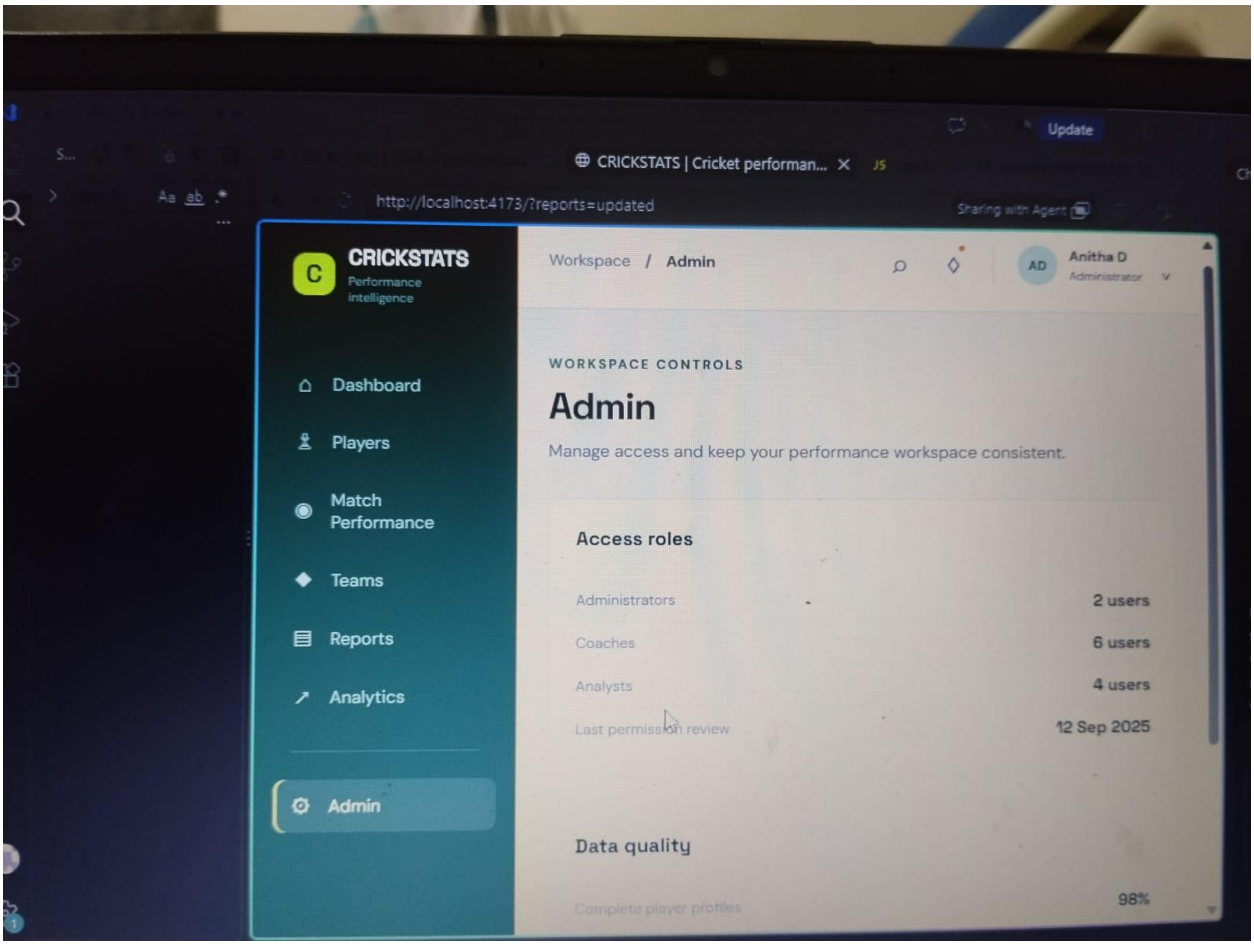
WON
LOST
DRAW
} <button type="button" class="secondary-btn" data-action="close-modal">Cancel</button><button class="primary-btn" type="submit">Save ${isPlayer?'player':'performance'}</button></div></form></div></div>;document.getElementById('entryForm').addEventListener('submit',e=>{e.preventDefault();document.getElementById('modalRoot').innerHTML='';toast(${isPlayer?'Player profile':'Match performance'} saved successfully);if(isPlayer){render('players')}else{render('performances')}})} document.addEventListener('submit',e=>{if(e.target.id!=='entryForm')return;const data=Object.fromEntries(new FormData(e.target));if(data.name){players.push({id:Date.now(),name:data.name,number:Number(data.number),state:data.state,nationality:'India',role:data.role,batting:data.batting,bowling:data.bowling,team:data.team,matches:0,runs:0,average:0,fours:0,sixes:0,wickets:0,catches:0,image:'https://i.pravatar.cc/100?u='+encodeURIComponent(data.name)})}else{const player=players.find(p=>p.name===data.player);performances.unshift({player:data.player,team:player?.team||'Unassigned',runs:Number(data.runs),balls:Number(data.balls),fours:Number(data.fours),sixes:Number(data.sixes),overs:Number(data.overs),wickets:Number(data.wickets),result:data.result,date:'18 Sep 2025'})}},{capture:true}); function toast(message){const t=document.createElement('div');t.className='toast';t.textContent=message;document.body.appendChild(t);setTimeout(()=>t.remove(),2800)} function bindViewEvents(){document.querySelectorAll('[data-view]:not(.nav-item)').forEach(el=>el.addEventListener('click',()=>render(el.dataset.view)));document.querySelectorAll('[data-action="open-report"]').forEach(el=>el.addEventListener('click',()=>{selectedReport=el.dataset.report;render('reportDetail')}));document.querySelectorAll('[data-action="back-reports"]').forEach(el=>el.addEventListener('click',()=>render('reports')));document.querySelectorAll('[data-action="export-report"]').forEach(el=>el.addEventListener('click',()=>toast(${selectedReport} export is ready)));document.querySelectorAll('[data-action="add-player"]').forEach(el=>el.addEventListener('click',()=>modal('player')));document.querySelectorAll('[data-action="add-performance"]').forEach(el=>el.addEventListener('click',()=>modal('performance')));document.querySelectorAll('[data-action="close-modal"]').forEach(el=>el.addEventListener('click',()=>document.getElementById('modalRoot').innerHTML=''));const search=document.getElementById('playerSearch');if(search){const filter=()=>{const

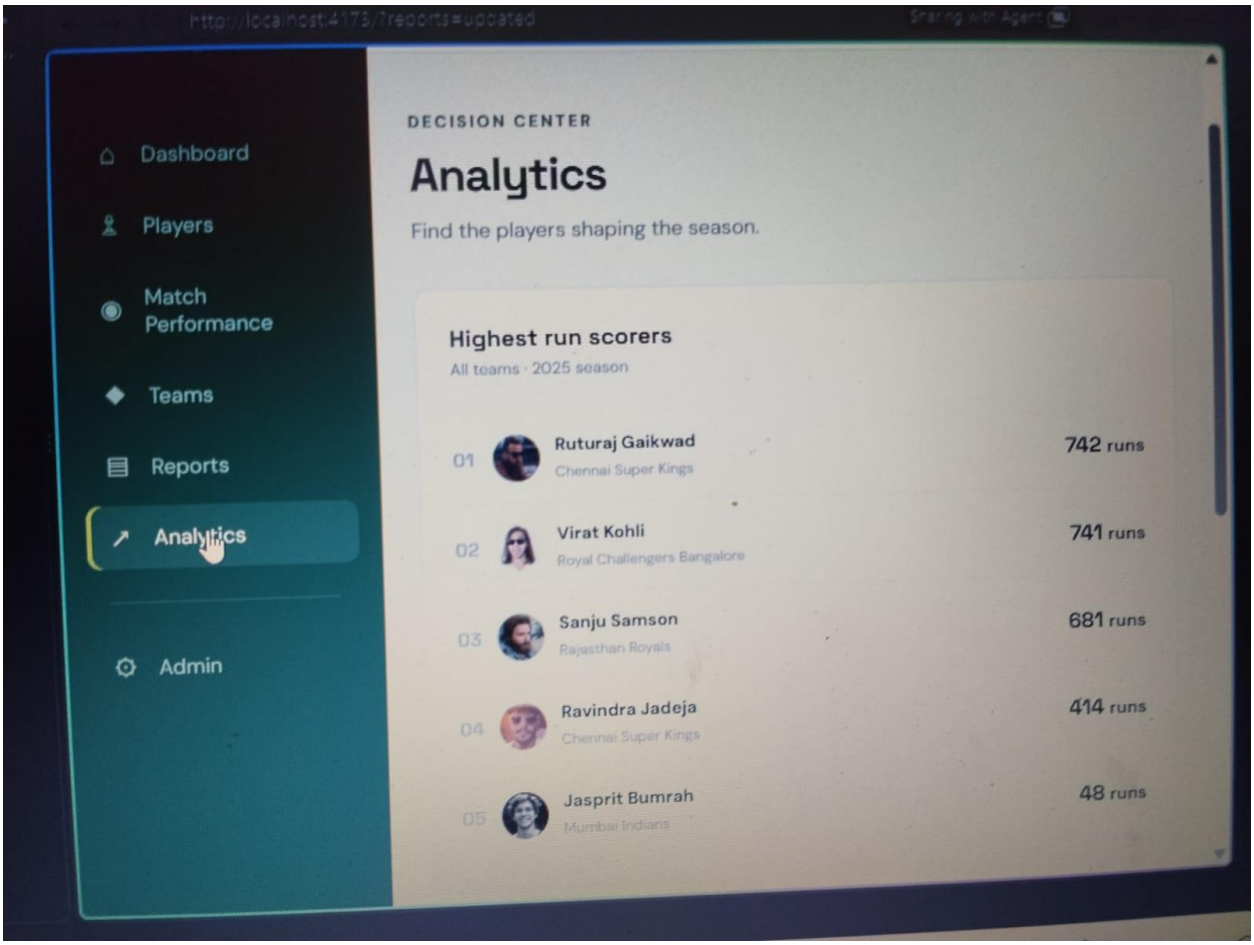
```
q=search.value.toLowerCase(),team=document.getElementById('teamFilter').value,role=document.getElementById('roleFilter').value;document.getElementById('playerRows').innerHTML=playerRows(players.filter(p=>(p.name.toLowerCase().includes(q)||p.team.toLowerCase().includes(q))&&(!team||p.team===team)&&(!role||p.role===role)))};search.addEventListener('input',filter);document.getElementById('teamFilter').addEventListener('change',filter);document.getElementById('roleFilter').addEventListener('change',filter)}const ps=document.getElementById('performanceSearch');if(ps)ps.addEventListener('input',()=>{const q=ps.value.toLowerCase();document.getElementById('performanceRows').innerHTML=performanceRows(performances.filter(p=>p.player.toLowerCase().includes(q)||p.team.toLowerCase().includes(q)))});} document.querySelectorAll('.nav-item').forEach(n=>n.addEventListener('click',()=>{render(n.dataset.view);document.getElementById('sidebar').classList.remove('open')}));document.getElementById('mobileMenu').addEventListener('click',()=>document.getElementById('sidebar').classList.toggle('open'));document.getElementById('modalRoot').addEventListener('click',e=>{if(e.target.id==='modalBackdrop')document.getElementById('modalRoot').innerHTML=''});render();
```

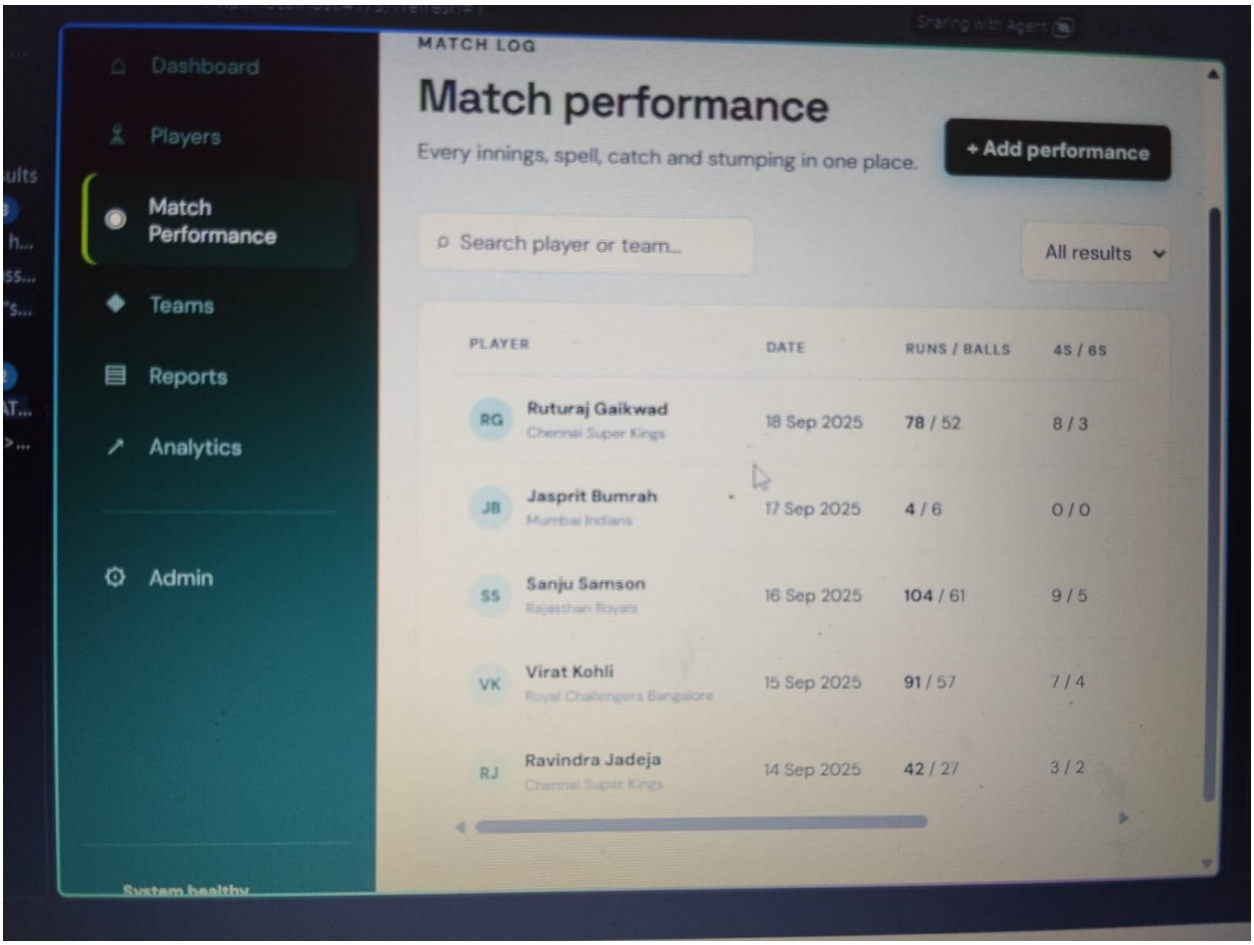
Output:

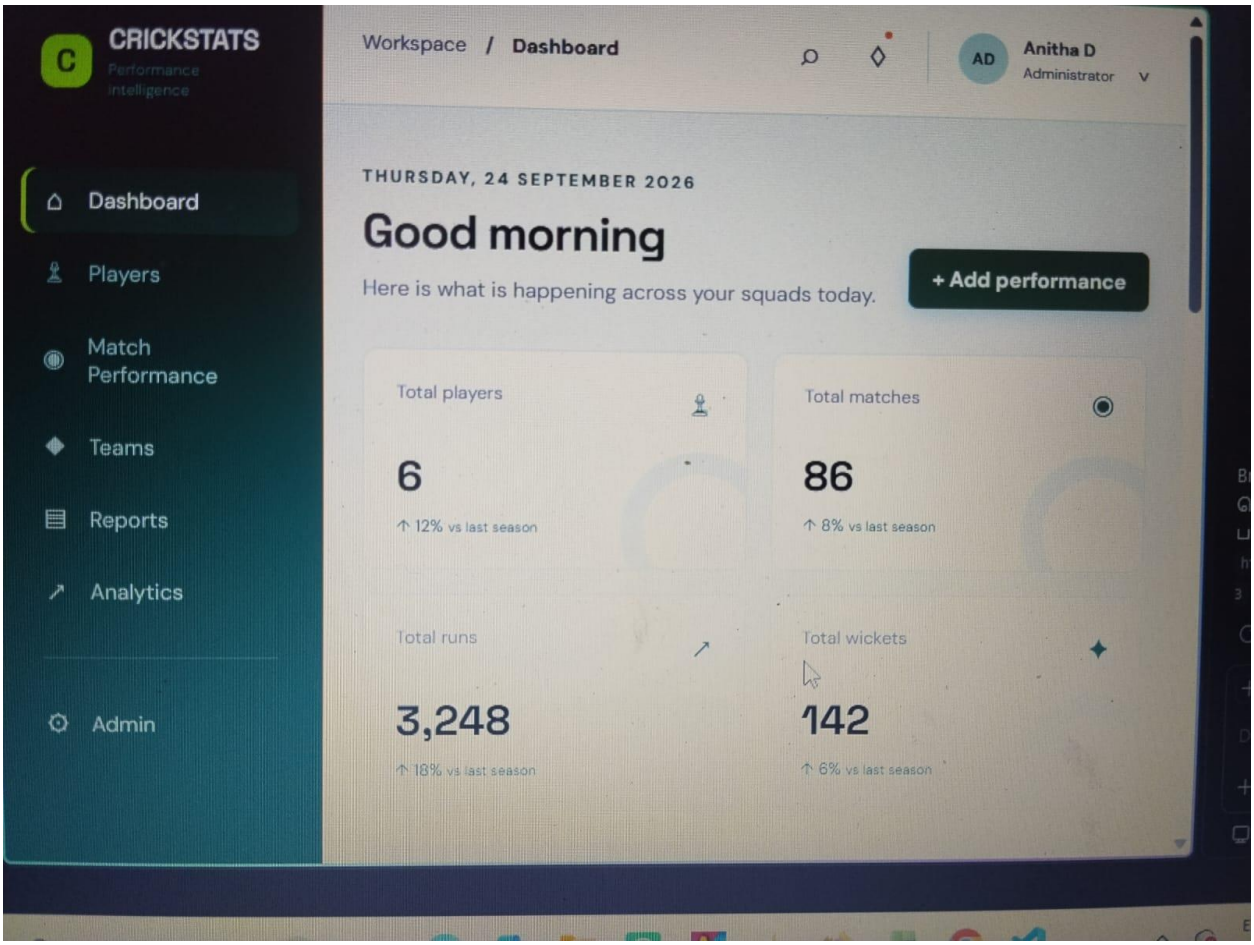

Dashboard
Players
Match Performance
Teams
Reports
Analytics
Admin
SQUAD DIRECTORY
Players
Manage player profiles and track individual career performance.
+ Add player
Search players...
All teams
All roles
PLAYER
TEAM
ROLE
MATCHES
Ruturaj Gaikwad
#31 · Maharashtra
Chennai Super Kings
18
Jasprit Bumrah
#93 · Gujarat
Mumbai Indians
Bowler
16
Sanju Samson
#9 · Kerala
Rajasthan Royals
Batsman & Wicket Keeper
17
Ravindra Jadeja
#8 · Gujarat
Chennai Super Kings
All Rounder
18

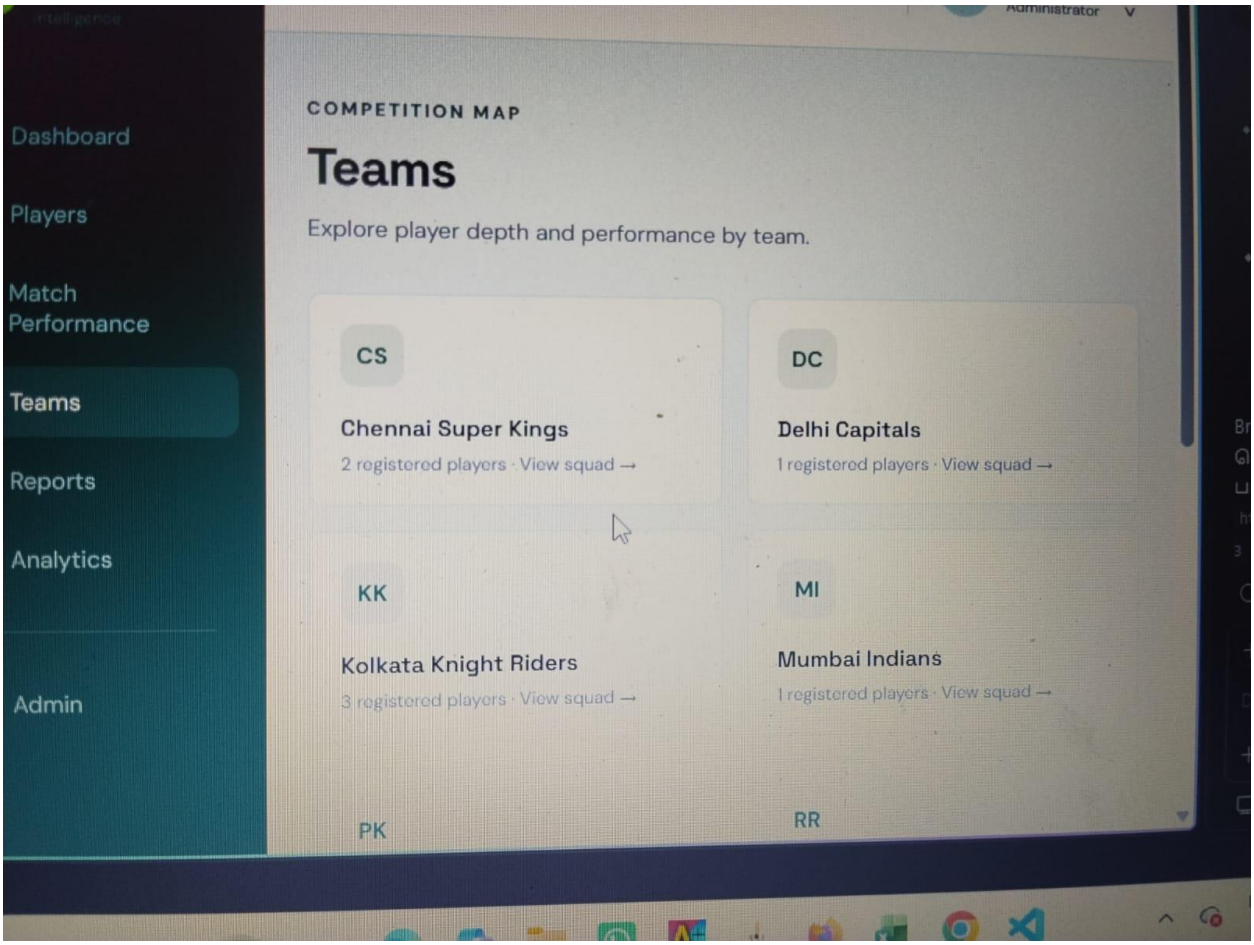

Dashboard
Players
Match Performance
Teams
Reports
Analytics
Admin
REPORTING SUITE
Reports
Ready-to-use performance reports for coaches and analysts.
Player Performance Report
A complete view of player output across batting, bowling and fielding.
Batting Report
Compare runs, averages, boundaries and strike contributions.
Bowling Report
Review overs, economy, wickets and maiden overs.
Team Report
Team-wise performance and squad depth.

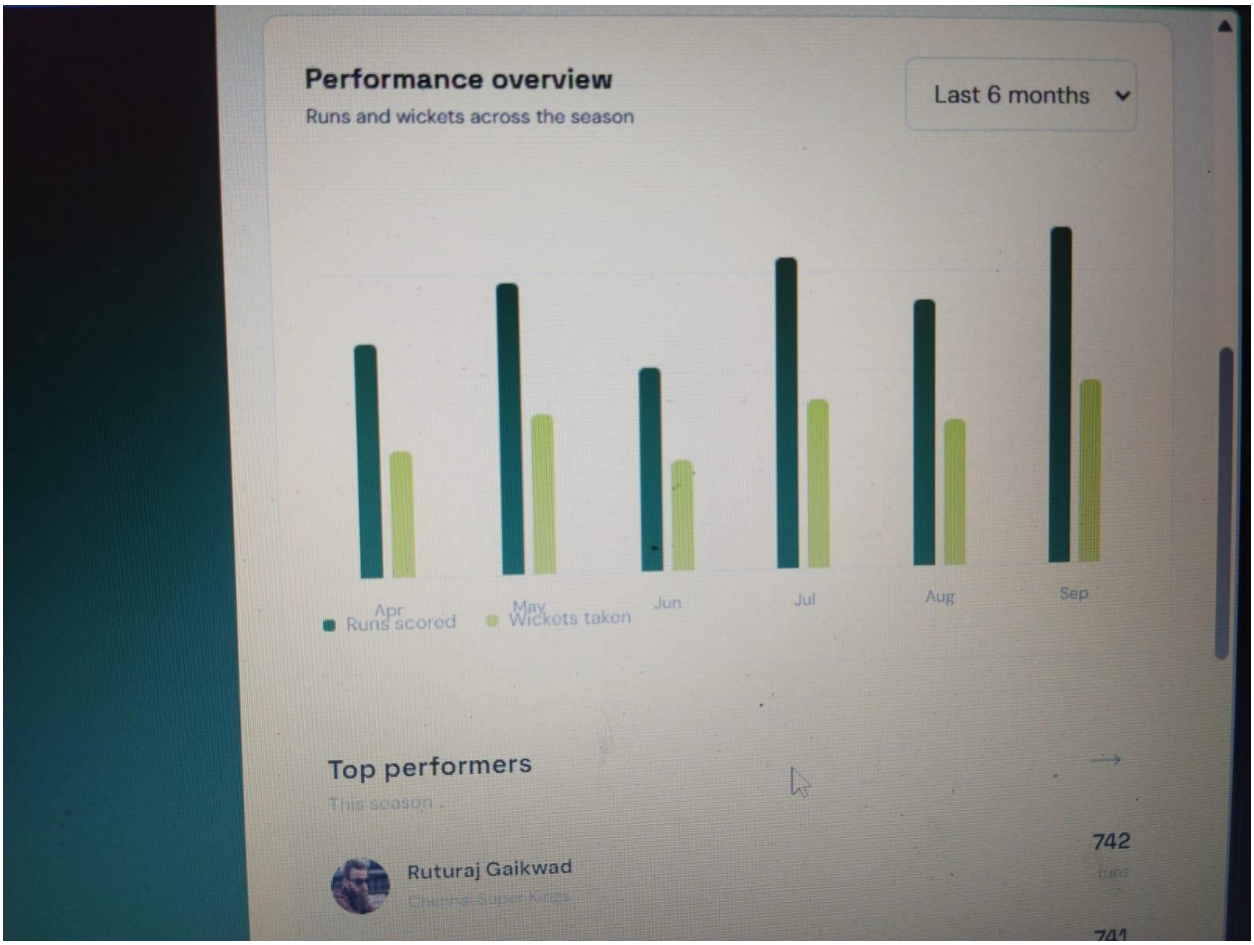

Update
CRICKSTATS | Cricket performan...
http://localhost:4173/?reports=updated
Sharing with Agent
CRICKSTATS
Performance intelligence
Workspace / Admin
AD
Anitha D
Administrator
Dashboard
Players
Match Performance
Teams
Reports
Analytics
Admin
WORKSPACE CONTROLS
Admin
Manage access and keep your performance workspace consistent.
Access roles
Administrators
2 users
Coaches
6 users
Analysts
4 users
Last permission review
12 Sep 2025
Data quality
Complete player profiles
98%


Dashboard
Players
Match Performance
Teams
Reports
Analytics
Admin
DECISION CENTER
Analytics
Find the players shaping the season.
Highest run scorers
All teams · 2025 season
01 Ruturaj Gaikwad Chennai Super Kings 742 runs
02 Virat Kohli Royal Challengers Bangalore 741 runs
03 Sanju Samson Rajasthan Royals 681 runs
04 Ravindra Jadeja Chennai Super Kings 414 runs
05 Jasprit Bumrah Mumbai Indians 48 runs


Dashboard
Players
Match Performance
Teams
Reports
Analytics
Admin
System healthy
MATCH LOG
Match performance
Every innings, spell, catch and stumping in one place.
+ Add performance
Search player or team...
All results
PLAYER
DATE
RUNS / BALLS
4S / 6S
RG
Ruturaj Gaikwad
Chennai Super Kings
18 Sep 2025
78 / 52
8 / 3
JB
Jasprit Bumrah
Mumbai Indians
17 Sep 2025
4 / 6
0 / 0
SS
Sanju Samson
Rajasthan Royals
16 Sep 2025
104 / 61
9 / 5
VK
Virat Kohli
Royal Challengers Bangalore
15 Sep 2025
91 / 57
7 / 4
RJ
Ravindra Jadeja
Chennai Super Kings
14 Sep 2025
42 / 27
3 / 2


CRICKSTATS
Performance intelligence
Workspace / Dashboard
AD
Anitha D
Administrator
Dashboard
Players
Match Performance
Teams
Reports
Analytics
Admin
THURSDAY, 24 SEPTEMBER 2026
Good morning
Here is what is happening across your squads today.
+ Add performance
Total players
6
↑ 12% vs last season
Total matches
86
↑ 8% vs last season
Total runs
3,248
↑ 18% vs last season
Total wickets
142
↑ 6% vs last season


Administrator
Dashboard
Players
Match Performance
Teams
Reports
Analytics
Admin
COMPETITION MAP
Teams
Explore player depth and performance by team.
CS
Chennai Super Kings
2 registered players · View squad →
DC
Delhi Capitals
1 registered players · View squad →
KK
Kolkata Knight Riders
3 registered players · View squad →
MI
Mumbai Indians
1 registered players · View squad →
PK
RR



10. FUTURE SCOPE

10.1 Future Enhancements

Future development can include mobile application integration, AI-based performance prediction, video analysis integration, player ranking, a public statistics portal, multi-league support, advanced predictive dashboards, automated notifications, and integrations with external cricket data providers.

11. APPENDIX

11.1 Salesforce Objects & Fields

Core objects: Player and Match Performance. Key relationship: Match Performance → Player through Master-Detail. Key calculated fields: AVG Score and roll-up summaries for matches, runs, fours, sixes, overs, wickets, stumpings, and catches.

11.2 Flow Configuration Summary

Player Flow contains a team selection screen, Player record retrieval filtered by the selected team, and a data table for displaying player performance information.

11.3 Suggested Evidence / Screenshots

Include screenshots of Salesforce Setup, Object Manager, Player object fields, Match Performance fields, relationship configuration, roll-up summary fields, CRICKSTATS Lightning App, Player records, Match Performance records, Flow Builder, reports, dashboards, and final test results.

12. REFERENCES

12.1 Official Documentation

Salesforce Developer Documentation and Trailhead resources should be used for Salesforce setup, custom objects, relationships, Flow, reports, dashboards, and Lightning App configuration. The official Salesforce documentation is available through the Salesforce Developer and Trailhead websites.